

Apuntes de Métodos Numéricos

Teoría + errores de implementación
reales, encontrados a la fuerza.
Se va completando en cada clase.

LOCALIZACIÓN DE RAÍCES

Bisección

Falsa posición

Punto fijo

Newton-Raphson

Secante

Raíz n-ésima

Plantear $f(x)=0$ (aplicaciones)

SISTEMAS LINEALES

Eliminación gaussiana

Jacobi

Gauss-Seidel

INTERPOLACIÓN

Lagrange

Coeficientes indeterminados

Cómo verificar el resultado

AJUSTE DE CURVAS

Regresión lineal

Coeficiente de correlación

Funciones arbitrarias

Regresión polinómica

Guía de programas (cuál usar)

Tipos de error (estimado vs. exacto)

Errores recurrentes en C++

Guía de programas agregada (parcial modelo) — hasta acá entra el primer parcial.

Resumen propio de la materia, ordenado por tema. Cada sección tiene la teoría base, el pseudocódigo o la fórmula tal como la vimos, y — lo más útil a la hora de programar — los errores concretos que fuimos pisando al pasar el pseudocódigo a C++. Se va a ir completando a medida que avancemos con dudas nuevas y temas nuevos.

CAPÍTULO 1 — LOCALIZACIÓN DE RAÍCES

Bisección

método cerrado

Parte de un intervalo $[a, b]$ donde $f(a) \cdot f(b) < 0$ (cambio de signo → por Bolzano hay una raíz adentro). En cada paso se calcula el punto medio y se descarta la mitad del intervalo que no cambia de signo.

PSEUDOCÓDIGO

hacer:

$c = (a + b) / 2$

 si $f(c) == 0 \rightarrow$ raíz exacta, listo

 si $f(a) \cdot f(c) < 0 \rightarrow b = c$

 si no $\rightarrow a = c$

mientras (error > tolerancia) Y (iter < max_iter)

Ejemplo paso a paso: primera iteración a mano

$f(x) = (x+1)/(x+4) - 0.25x$, intervalo $[a, b] = [1.75, 3.2]$ (acá c_{viejo} arranca en b , no en a — así lo pedía este enunciado puntual, revisar siempre qué pide el criterio de error de cada ejercicio).

$$c = (a+b)/2 = (1.75+3.2)/2 = 2.4750000000$$

$$f(a) = f(1.75) = 0.0407608696$$

$$f(c) = f(2.475) = -0.0820704633$$

$$f(a) \cdot f(c) < 0 \rightarrow \text{la raíz está entre } a \text{ y } c \rightarrow b = c$$

$$\begin{aligned} \text{error} &= |c - c_{\text{viejo}}| / (\tfrac{1}{2} \cdot |c + c_{\text{viejo}}|) \times 100 && (c_{\text{viejo}} = b = 3.2) \\ &= |2.475 - 3.2| / (\tfrac{1}{2} \cdot |2.475 + 3.2|) \times 100 \\ &= 0.725 / 2.8375 \times 100 \\ &= 25.5506607930 \% \end{aligned}$$

Nuevo intervalo: [1.75, 2.475]

Cuántas iteraciones hacen falta (Problema 4)

Cada bisección acota el error con $|r - c| \leq (b-a)/2$ sobre el intervalo vigente. Como el intervalo se parte a la mitad en cada paso, después de n bisecciones sucesivas:

$$|r - c_n| \leq \frac{b - a}{2^n}$$

Pidiendo que ese error sea menor que un δ elegido y despejando n :

$$n = \lceil \log_2((b-a) / \delta) \rceil = \lceil \frac{\ln((b-a)/\delta)}{\ln 2} \rceil$$

Se redondea siempre **hacia arriba** (ceiling), porque n tiene que ser entero y la desigualdad se tiene que seguir cumpliendo.

VERIFICADO CON DATOS REALES

[0,1], $\delta=1e-4 \rightarrow n=\lceil 13.29 \rceil=14$ iteraciones. [60,65], $\delta=1e-5 \rightarrow n=\lceil 18.93 \rceil=19$ iteraciones. Las dos coinciden exactamente con lo que tardó el programa corriendo de verdad.

ATENCIÓN AL PROGRAMAR

- `c_viejo` tiene que inicializarse (`c_viejo = a`) *antes* del loop. Si no, el primer `error = |c - c_viejo|` usa memoria basura.

- La condición del do-while es $\&\&$, no $||$: "seguir mientras no convergió **y** no me pasé del máximo". Con $||$ el tope de iteraciones queda anulado — riesgo de loop infinito si el método no converge.
- Si el intervalo es simétrico y la raíz cae justo en el punto medio (ej. $a^{10}-1$ en $[0,2]$, raíz=1), bisección puede converger en **1 sola** iteración. Es correcto, pero no es representativo del caso general — no sirve para comparar velocidades de convergencia.

ELEGIR BIEN EL INTERVALO: CUIDADO CON RAÍCES ESPURIAS

Cuando la ecuación sale de despejar/manipular otra ecuación original (pasar una fracción a la forma $f(x)=0$, por ejemplo), a veces aparecen raíces "triviales" que matemáticamente cierran pero no tienen sentido físico. Antes de elegir $[a, b]$, conviene graficar la función completa y armar el intervalo para que aísle **solo** la raíz que buscás.

Caso real (Problema 5, ciclo termodinámico): despejando $\eta(x)=0.3$ con $x=T_2/T_1$ se llega a $(1.45/0.7) \cdot (1-1/x) - \ln(x) = 0$. Esa función tiene una raíz espuria en $x \approx 1$ (ahí $T_2=T_1$, no hay ciclo real — numerador y denominador de η se anulan juntos) y la raíz física de interés bastante más lejos, en $x \approx 5.41$. Usar $[4, 6]$ como intervalo evita agarrar la de $x=1$ por accidente.

Falsa posición

método cerrado

También parte de $[a, b]$ con cambio de signo, pero en vez de partir al medio, traza la recta secante entre $(a, f(a))$ y $(b, f(b))$ y usa el cruce con el eje x como aproximación:

$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$

Actualiza a o b igual que bisección, según el signo de $f(a) \cdot f(c)$.

Ejemplo paso a paso: primera iteración a mano

$f(x) = \ln(x^2+1) - \sin(x)$, intervalo $[a,b]=[1, 2]$ (cambio de signo: $f(1)<0$, $f(2)>0$).

$$f(1) = \ln(2) - \sin(1) = -0.1483238042$$

$$f(2) = \ln(5) - \sin(2) = 0.7001404856$$

$$\begin{aligned} c &= (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a)) \\ &= (1 \cdot 0.7001404856 - 2 \cdot (-0.1483238042)) / (0.7001404856 - (-0.1483238042)) \\ &= 0.9967880940 / 0.8484642898 \\ &= 1.1748144336 \end{aligned}$$

$$f(c) = f(1.1748144336) = -0.0554384022$$

$$f(a) \cdot f(c) > 0 \quad (\text{los dos negativos}) \quad \rightarrow \quad a = c$$

Nuevo intervalo: $[1.1748144336, 2]$

NO SIEMPRE ES MÁS RÁPIDA QUE BISECCIÓN

Cuando la función es muy asimétrica en curvatura cerca de la raíz, un extremo del intervalo se puede quedar "pegado" y casi no avanza — convergencia lineal muy lenta.

Caso real: $g(a) = a^{10} - 1$ en $[0,2]$ necesitó **1147 iteraciones** (bisección hizo 1, por el caso particular de arriba). Moraleja: no asumir que un método "más inteligente" siempre gana — depende de la forma de la función.

MISMOS CUIDADOS DE IMPLEMENTACIÓN QUE BISECCIÓN

$c_{\text{viejo}} = a$ antes del loop, condición $\&\&$ en el while, y ojo con el max_iter : si es muy chico, puede cortar el método antes de que termine de converger (no es que esté "trabado", solo hace falta más margen).

Punto fijo método abierto

Se reescribe $f(x)=0$ como $x = g(x)$ y se itera $x_{k+1} = g(x_k)$ a partir de una única semilla x_0 .

CONDICIÓN DE CONVERGENCIA

$$|g'(x)| < 1 \quad \text{cerca de la raíz}$$

NOTA

Chequear g' una sola vez en x_0 (con diferencia finita, $(g(x_0+h)-g(x_0))/h$) es una simplificación razonable, pero solo es una pista inicial — no garantiza que $|g'| < 1$ se mantenga en todo el recorrido de la iteración.

Ejemplo paso a paso: primera iteración a mano

$g(x) = \sqrt{(x+5)/2}$, semilla $x_0 = 0$.

$$g'(x) = 1 / (4 \cdot g(x)) \quad (\text{regla de la cadena sobre } g(x) = ((x+5)/2)^{(1/2)})$$

$$g(0) = \sqrt{5/2} = 1.5811388301$$

$$g'(0) = 1 / (4 \cdot 1.5811388301) = 0.1581138830$$

$$|g'(0)| = 0.1581138830 < 1 \quad \rightarrow \quad \text{se cumple el criterio, converge}$$

$$x_1 = g(x_0) = g(0) = 1.5811388301$$

$$\text{error} = |x_1 - x_0| = 1.5811388301$$

Cuándo NO converge (ejemplo real)

Ojo con la diferencia entre **puntos fijos de g** (resolver $x=g(x)$) y **raíces de g** (resolver $g(x)=0$) — son ecuaciones distintas, es fácil despejar de una pensando en la otra.

Con $g(x)=x^2+x-4$, la ecuación $x=g(x)$ da $x^2-4=0$, o sea puntos fijos $P=2$ y $P=-2$. Con $g'(x)=2x+1$:

$$g'(2) = 5 \quad g'(-2) = -3$$

$|g'(P)|$ da mayor que 1 en **los dos** puntos fijos $\rightarrow$ la iteración directa $x_{k+1}=g(x_k)$ diverge, sin importar qué tan cerca arranque la semilla:

SIMULACIÓN ARRANCANDO EN $x_0=2.01$ (A 0.01 DEL PUNTO FIJO)

$$x_0 = 2.010000$$

$$x_1 = 2.050100$$

x2 = 2.253010
x3 = 3.329064
x4 = 10.411732
x5 = 114.815895 ← se va al horno

TRAMPA COMÚN

Despejar x de otra forma (ej. $x=4-x^2$) puede parecer una alternativa, pero eso despeja $g(x)=0$ (sus raíces, ≈ 1.56 y ≈ -2.56), **no** los puntos fijos de $x=g(x)$. Es una ecuación distinta — y en este caso tampoco converge ($\phi'(x)=-2x$, también mayor que 1 en esas raíces).

La ventaja de $g'(P) \approx 0$: de convergencia lineal a cuadrática

Reacomodando la *misma* ecuación $x^2=4$ de otra forma — $\psi(x) = (x + 4/x)/2$ (el promedio; matemáticamente es Newton-Raphson aplicado a $x^2-4=0$) — la derivada da:

$$\psi'(x) = 1/2 - 2/x^2 \quad \psi'(2) = 0 \quad \psi'(-2) = 0$$

CONVERGENCIA ARRANCANDO EN $x_0=3$ (LEJOS DEL PUNTO FIJO)

x0 = 3.0000000000
x1 = 2.1666666667
x2 = 2.0064102564
x3 = 2.0000102400
x4 = 2.0000000000 ← precisión de máquina en 4 pasos

CONCLUSIÓN

Cuando $0 < |g'(P)| < 1$, converge pero **linealmente** (el error se reduce por un factor $\approx |g'(P)|$ en cada paso). Cuando $g'(P) \approx 0$, pasa a converger **cuadráticamente** — el error se eleva al cuadrado en cada paso en vez de multiplicarse por una constante. Es la misma razón por la que Newton-Raphson converge tan rápido: NR es una iteración de punto fijo con $\phi(x)=x-f(x)/f'(x)$, y en una raíz simple siempre da $\phi'(P)=0$.

Newton-Raphson

método abierto

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Converge cuadráticamente (mucho más rápido que los métodos cerrados) cuando converge, pero necesita la derivada analítica y una buena semilla.

ATENCIÓN

Revisar que $|f'(x_k)|$ no sea ~ 0 antes de dividir — si no, el paso explota o se vuelve inestable.

Ejemplo paso a paso: primera iteración a mano

$f(x) = x^x - 5$, $f'(x) = x^x \cdot (\ln(x) + 1)$, semilla $x_0 = 1$.

$$f(1) = 1^1 - 5 = -4$$

$$f'(1) = 1^1 \cdot (\ln(1) + 1) = 1 \cdot (0 + 1) = 1$$

$$x_1 = x_0 - f(x_0)/f'(x_0) = 1 - (-4)/1 = 5$$

$$\text{error} = |x_1 - x_0| = |5 - 1| = 4$$

Secante

método abierto

Igual que Newton, pero aproxima la derivada con diferencia finita entre los dos últimos puntos — no hace falta la derivada analítica, pero se necesitan 2 semillas.

$$x_{k+1} = x_k - \frac{f(x_k) \cdot (x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}$$

ATENCIÓN

Revisar que $f(x_k) - f(x_{k-1})$ no sea ~ 0 antes de dividir.

Ejemplo paso a paso: primera iteración a mano

$f(x) = x^2 - 9x + 2$, semillas $x_0=0$, $x_1=1$.

$$f(x_0) = f(0) = 0 - 0 + 2 = 2$$

$$f(x_1) = f(1) = 1 - 9 + 2 = -6$$

$$x_2 = x_1 - f(x_1) \cdot (x_1 - x_0) / (f(x_1) - f(x_0))$$

$$= 1 - (-6) \cdot (1 - 0) / (-6 - 2)$$

$$= 1 - (-6) / (-8)$$

$$= 1 - 0.75$$

$$= 0.25$$

$$\text{error} = |x_2 - x_1| = |0.25 - 1| = 0.75$$

Raíz n-ésima

Newton-Raphson especializado

Newton-Raphson aplicado directo a $f(x) = x^n - A$ (buscar la raíz n-ésima de A) se simplifica a una fórmula cerrada — no hace falta calcular f ni f' por separado en cada paso.

N=2 (RAÍZ CUADRADA)

$$p_k = \frac{1}{2} \cdot (p_{k-1} + A/p_{k-1})$$

N=3 (RAÍZ CÚBICA)

$$p_k = (2 \cdot p_{k-1} + A/p_{k-1}^2) / 3$$

Ejemplo paso a paso: primera iteración a mano

$A=10$, $n=2$ (raíz cuadrada de 10), semilla $p_0=3$.

$$\begin{aligned}
 p_1 &= 0.5 \cdot (p_0 + A/p_0) \\
 &= 0.5 \cdot (3 + 10/3) \\
 &= 0.5 \cdot 6.3333333333 \\
 &= 3.1666666667
 \end{aligned}$$

$$\text{error} = |p_1 - p_0| = 0.1666666667$$

(valor exacto: $\sqrt{10} = 3.1622776602$ — a una sola iteración ya está a 0.0044 de distancia)

Plantear $f(x)=0$ a partir de un problema

A veces no te dan la ecuación servida — hay que armarla a partir de la geometría o la física del enunciado. Tres ejemplos reales donde hubo que derivarla desde cero.

Contenedor sin tapa (recortar esquinas)

De una hoja de 10×16 , se recorta un cuadrado de lado x en cada esquina y se doblan los bordes: queda una caja sin tapa con altura= x , largo= $16-2x$, ancho= $10-2x$. Volumen: $V(x) = x(16-2x)(10-2x)$. Igualando a un volumen pedido y desarrollando:

$$x(16-2x)(10-2x) = 100 \rightarrow f(x) = x^3 - 13x^2 + 40x - 25 = 0$$

FILTRO FÍSICO

3 raíces matemáticas: 0.839, 3.402, 8.759. Como hace falta $10-2x > 0$, el dominio válido es $0 < x < 5$ — la raíz 8.759 queda afuera (da ancho negativo). Dos soluciones físicamente válidas: $x \approx 0.839$ y $x \approx 3.402$.

Catenaria (cable colgante)

El enunciado ya da la ecuación $b = C \cdot \cosh(a/C) - C$ (incógnita C), solo hay que pasarla a la forma $f(C)=0$:

$$f(C) = C \cdot \cosh(a/C) - C - b = 0$$

VERIFICADO

Con $a=12$, $b=5$: $C \approx 15.167028341664$.

Esfera flotando (principio de Arquímedes)

Acá no viene ninguna ecuación en el enunciado — sale de igualar el peso de la esfera con el peso del agua desplazada (densidad del agua = 1 gr/cm³):

VOLUMEN DEL CASQUETE ESFÉRICO SUMERGIDO, ALTURA d , ESFERA DE RADIO r

$$V_{\text{casquete}}(d) = \frac{\pi \cdot d^2}{3} \cdot (3r - d)$$

$$\rho \cdot \frac{4}{3} \pi r^3 = \frac{\pi \cdot d^2}{3} (3r - d) \quad \rightarrow \quad f(d) = d^3 - 3r \cdot d^2 + 4\rho r^3 = 0$$

VERIFICADO

Con $r=10$, $\rho=0.638$: $f(d)=d^3-30d^2+2552=0$. Raíces: 26.31, 11.86, -8.18.

Filtro físico: $0 < d < 2r = 20$ (la profundidad sumergida no puede superar el diámetro) → única válida: $d \approx 11.8615$ cm.

EL PATRÓN SE REPITE

Mismo tema que en el Problema 5 de bisección (el ciclo termodinámico, más arriba): plantear la ecuación no alcanza — casi siempre hay que **filtrar** las soluciones matemáticas según el dominio físico real de la variable (un ancho no puede ser negativo, una profundidad no puede superar el diámetro, etc.). Conviene siempre preguntarse qué rango tiene sentido *antes* de mirar las raíces.

Triangulación

PSEUDOCÓDIGO (1-INDEXADO, COMO LO DA LA PROFESORA)

```
Do i = 1,...,n-1      ← Pivoteo
  Do j = i+1,...,n
    factor = -a_ji / a_ii
    Do k = i+1,...,n
      a_jk = a_ik·factor + a_jk
    end Do
    b_j = b_i·factor + b_j
  end Do
end Do
```

Determinante

Producto de la diagonal ya triangulada: $\det = a_{11} \cdot a_{22} \cdot \dots \cdot a_{nn}$. Si $\det = 0 \rightarrow$ matriz singular, no tiene solución única.

Retrosustitución

$$x_i = \frac{b_i - \sum_{j>i} a_{ij} \cdot x_j}{a_{ii}} \quad (\text{empezando por } x_n \text{ y bajando hasta } x_1)$$

EL ERROR QUE MÁS SE REPITE EN TODO EL CURSO

El pseudocódigo de la profesora es **1-indexado** (arranca en 1, a_{11} es el primer elemento). Los arrays de C++ son **0-indexados**. Hay que correr los puntos de *arranque* de los loops un lugar ($i=1 \rightarrow i=0$, $i=2 \rightarrow i=1$, $a[1][1] \rightarrow a[0][0]$) — pero ojo: los límites superiores ($i < n-1$, $j < n$, etc.) casi siempre ya quedan bien escritos tal cual, sin tocar nada.

Interpolación polinómica = sistema de Vandermonde

"Encontrar el polinomio $y = A + Bx + Cx^2 + Dx^3$ que pasa por 4 puntos dados" no es un tema aparte — es una aplicación directa de resolver un sistema lineal. Por cada punto (x_i , y_i) conocido, se arma una ecuación evaluando el polinomio genérico ahí: $A + B \cdot x_i + C \cdot x_i^2 + D \cdot x_i^3 = y_i$. Con 4 puntos, quedan 4 ecuaciones para las 4 incógnitas A, B, C, D — se resuelve con Gauss/Gauss-Jordan como cualquier otro sistema. La matriz que se arma (filas $[1, x_i, x_i^2, x_i^3]$) se llama **matriz de Vandermonde**.

TRAMPA: EL COEFICIENTE DE A SIEMPRE ES 1

El coeficiente de A en cada fila es **siempre 1**, sin importar el valor de x_i — ni siquiera se anula en $x_i=0$ (lo que se anula ahí son los términos Bx , Cx^2 , Dx^3 , no el de A, que no depende de x). Un error común es poner 0 en esa columna para las filas donde no es el punto "especial" — eso arma mal el sistema.

Lo traicionero: si el punto $(0, y_0)$ tiene $y_0=0$, la solución para A da 0, y ahí $0 \cdot A$ y $1 \cdot A$ dan lo mismo (cero) — el error queda "tapado" y el resultado sale bien por casualidad. Si A hubiese dado cualquier otro valor, el error sí cambia la respuesta (lo comprobamos forzando $A=5$: con el `0` mal puesto daba $B, C, D = -0.167, 1.5, -0.333$; con la matriz bien armada daba $-9.33, 6.5, -1.17$ — completamente distinto).

Pivoteo parcial

Si $|a_{ii}|$ da muy chico, buscar en esa columna la fila con mayor valor absoluto y swapear filas (y también b) antes de seguir.

DOS BUGS REALES ENCONTRADOS ACÁ

- Guardar el **valor absoluto** ($fabs(\dots)$) al comparar y al actualizar el "mayor valor" — si se guarda el valor con signo, un negativo grande puede perder la comparación contra un positivo chico.
- Al swapear dos filas, mover **todas** las columnas (de 0 a $n-1$) — arrancar el loop del swap en 1 en vez de 0 deja la columna 0 sin intercambiar (mismo error de indexado de arriba, escondido en otro lado del código).

Gauss vs. Gauss-Jordan

Dan siempre el mismo resultado (verificado) — la diferencia es cuánto se triangula:

- **Gauss:** ceros solo por debajo de la diagonal, después retrosustitución (despejar desde x_n hacia atrás).
- **Gauss-Jordan:** además elimina por **encima** de la diagonal (columna pivote de atrás para adelante) y normaliza cada fila — queda $[I | x]$, se lee la solución directo, sin retrosustitución. Hace más cuentas que Gauss (~50% más), pero si el enunciado pide Gauss-Jordan específicamente, hay que mostrar esa eliminación extra, no alcanza con el resultado triangulado + retrosustitución.

CUIDADO CON EL DETERMINANTE EN GAUSS-JORDAN

El determinante hay que calcularlo **apenas termina la triangulación hacia abajo** (producto de la diagonal), antes de normalizar filas. Si se normaliza primero (dividir cada fila por su pivote para dejar la diagonal en 1) y se calcula el determinante después, va a dar siempre 1 — mal.

OTRO CUIDADO CON EL DETERMINANTE: EL SIGNO CAMBIA CON CADA SWAP

Cada vez que el pivoteo intercambia dos filas de verdad, el signo del determinante real se invierte — si no se compensa, el producto de la diagonal final da el **valor absoluto correcto pero el signo equivocado** cuando hubo una cantidad impar de swaps. Se arregla con una variable `signoDet` (arranca en 1, se multiplica por -1 en cada intercambio real) que multiplica al producto de la diagonal.

Este bug estuvo escondido varios ejercicios: como los sistemas anteriores no necesitaban pivotear de verdad, el código daba bien "por accidente". Recién se destapó en un ejercicio que *obligaba* a pivotear (enunciado: $a_{11}=4$, a_{22} queda en 0 después de triangular, hace falta swap) — el determinante daba -4 en vez de +4. Buena lección general: un bug en una rama de código que nunca se ejecuta no se nota hasta que un caso de prueba fuerza justo esa rama.

Matrices mal condicionadas

A veces el sistema en sí es "frágil": un cambio mínimo en un dato de entrada produce un cambio enorme en la solución. Se detecta comparando el determinante contra la **norma de Frobenius** de la matriz:

$$\|A\| = \sqrt{(\sum_{i=1}^n \sum_{j=1}^n A_{ij}^2)}$$

Si $\det(A)$ da chico **comparado con $\|A\|$** , la matriz es "cuasi-singular" — mal condicionada. No hay un umbral fijo, es una comparación de órdenes de magnitud.

CASO REAL — SISTEMA DE INTERPOLACIÓN CON MATRIZ MAL CONDICIONADA

Cambiando **una sola celda** de la matriz ($A_{1,1}$) apenas un 0.005% (20514→20515), sin tocar nada más:

	c1,c2,c3,c4	det(A)	A
original:	1, 0, 0, 0	144	21518.5
A11=20515:	0.642857, 3.75, -12.3929, 12.75	224	21519.5

$\|A\|$ casi no se movió, pero $\det(A)$ cambió un 55% y la solución cambió por completo. Esa sensibilidad extrema a una perturbación mínima es justamente la señal de que la matriz está mal condicionada — el determinante es insignificante comparado con la norma.

OJO AL CALCULAR $\|A\|$ EN EL CÓDIGO

Hay que calcularla con la matriz **original**, antes de triangular — `triangular()` pisa las entradas de abajo de la diagonal con ceros, así que si se calcula después, la norma sale mal (le faltan términos).

Método de Jacobi iterativo

Para resolver $Ax=b$: se despeja cada x_i de su propia ecuación, usando siempre los valores de **la iteración anterior** (a diferencia de Gauss-Seidel, que ya usa los valores nuevos apenas se calculan).

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} \cdot x_j^{(k)}}{a_{ii}}$$

Tres cosas para implementarlo

1. Asegurar $a_{ii} \neq 0$ — se garantiza pidiendo que la matriz sea **diagonalmente dominante**: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ en cada fila. Condición *suficiente, no necesaria* (puede converger igual sin serlo).
2. Inicializar el vector $X^{(0)} = (0, 0, \dots, 0)$.
3. Criterio de corte por tolerancia, en base al error.

Error

$$e = \|X_{\text{nuevo}} - X_{\text{viejo}}\| \quad (\text{norma euclídea} \rightarrow \text{error absoluto})$$

$$E = \frac{e}{\|X_{\text{nuevo}}\|} \times 100 \quad (\text{error porcentual})$$

Ejemplo paso a paso: primera iteración a mano

Sistema $3x_1 + x_2 = 10$, $-x_1 + 5x_2 = 18$, $x_0 = (0, 0)$ (solución exacta: $(2, 4)$).

```
x1 = (10 - x2_viejo) / 3 = (10-0)/3 = 3.3333333333
x2 = (18 + x1_viejo) / 5 = (18+0)/5 = 3.6000000000 ← usa x1_VIEJO (0), no el rec

error estimado = \|Xnuevo-Xviejo\| =  $\sqrt{(3.3333333333^2 + 3.6^2)}$  = 4.9062314
error exacto   = \|Xnuevo-Xexacto\| =  $\sqrt{((3.3333333333-2)^2 + (3.6-4)^2)}$  = 1.3920408
```

BUG ENCONTRADO: LA SUMA NO SE RESETEA POR FILA

suma tiene que volver a 0 en **cada fila** i (adentro del loop de i , antes del loop de j) — no una sola vez por iteración global. Si no, se arrastra entre filas y la fórmula de Jacobi queda mal.

BUENA PRÁCTICA

Guardar el error de la iteración anterior y, si el error nuevo es *mayor* que el anterior, cortar con un aviso de "el método no converge" — Jacobi no siempre converge, y así se detecta a tiempo en vez de iterar al pedo.

Gauss-Seidel

iterativo

Igual que Jacobi, pero usando el valor de x_j ya actualizado en la misma pasada apenas se calcula, en vez de esperar a la iteración siguiente. Por eso converge bastante más rápido que Jacobi cuando converge.

$$x_i^{\text{nuevo}} = \frac{b_i - \sum_{j < i} a_{ij} \cdot x_j^{\text{nuevo}} - \sum_{j > i} a_{ij} \cdot x_j^{\text{viejo}}}{a_{ii}}$$

TRUCO DE IMPLEMENTACIÓN

No hace falta mantener dos arrays separados (Xviejo/Xnuevo) como en Jacobi — alcanza con **un solo array X actualizado en el lugar**. Cuando se calcula $X[i]$, las posiciones $X[0 \dots i-1]$ ya tienen los valores nuevos de esta pasada, y $X[i+1 \dots n-1]$ todavía tienen los viejos (no se tocaron todavía) — exactamente lo que pide la fórmula, sin necesitar ningún caso especial para $i=1$. Eso sí, conviene guardar una copia de X de *antes* de la pasada (Xviejo) solo para poder calcular el error al final.

Ejemplo paso a paso: primera iteración a mano

Mismo sistema que en Jacobi (para comparar directo): $3x_1 + x_2 = 10$, $-x_1 + 5x_2 = 18$, $x_0 = (0, 0)$.

```
x1 = (10 - x2_viejo) / 3 = (10-0)/3 = 3.3333333333
x2 = (18 + x1_NUEVO) / 5 = (18+3.3333333333)/5 = 4.2666666667 ← ya usa el x1 rec
error estimado = ||Xnuevo-Xviejo|| = √(3.3333333333² + 4.2666666667²) = 5.4
error exacto = ||Xnuevo-Xexacto|| = √((3.3333333333-2)² + (4.2666666667-4)²) = 1.35
```

VERIFICADO: CONVERGE MUCHO MÁS RÁPIDO QUE JACOBI

Mismo sistema DD ($\begin{bmatrix} 3 & 1 & 1 \\ 0 & 2 & 1 \\ 1 & 3 & 5 \end{bmatrix}$, $b = [5, 3, 9]$), misma tolerancia:

```
Jacobi:      25 iteraciones
Gauss-Seidel: 9 iteraciones
```

Las dos convergen a la misma solución $(1, 1, 1)$.

Gauss-Seidel con relajación

Después de calcular x_i con la fórmula normal de Gauss-Seidel, se mezcla con el valor viejo:

$$X_n[i] = w \cdot X_n[i] + (1-w) \cdot X_v[i] \quad w \in [0,2]$$

- $w=1 \rightarrow$ Gauss-Seidel tradicional (sin cambios).
- $w \in (0,1) \rightarrow$ **sub-relajación**: sirve para hacer converger un sistema que de otra forma no converge.
- $w \in (1,2] \rightarrow$ **sobre-relajación**: le da más peso a los valores nuevos, para acelerar un método que ya converge (no siempre ayuda — depende de la matriz).

NO HAY QUE CALCULAR W

La elección del w óptimo es específica de cada problema y se determina empíricamente — para los ejercicios de la materia, si hace falta relajación, w viene dado en el enunciado, no hay que calcularlo.

El orden de las filas importa

A diferencia de eliminación gaussiana (donde el pivoteo reordena filas libremente sin cambiar si el sistema tiene solución), Jacobi y Gauss-Seidel **no son invariantes al orden de las ecuaciones**. El mismo sistema, reescrito con las filas en otro orden, puede pasar de converger rapidísimo a divergir sin remedio.

CASO REAL: MISMO SISTEMA, DOS ÓRDENES DE FILAS

Orden 1 (diagonalmente dominante): $4x - y + z = 7$, $4x - 8y + z = -21$, $-2x + y + 5z = 15 \rightarrow$ Jacobi converge en 13 iteraciones, Gauss-Seidel en 8. Los dos dan $x=2$, $y=4$, $z=3$.

Orden 2 (mismas 3 ecuaciones, filas 1 y 3 intercambiadas): deja de ser DD en 2 de las 3 filas $\rightarrow$ ninguno de los dos métodos converge.

Para comprobar que diverge de verdad (no es solo el corte conservador del programa), dejando correr la iteración sin cortar, $\|x\|$ crece sin límite y con signo oscilante:

Jacobi: 10.6 $\rightarrow$ 41 $\rightarrow$ 100 $\rightarrow$ 362 $\rightarrow$ 960 $\rightarrow$ 3496 $\rightarrow$... $\rightarrow$ 3.1×10^6
 Gauss-Seidel: 36.7 $\rightarrow$ 288 $\rightarrow$ 2415 $\rightarrow$ 20140 $\rightarrow$... $\rightarrow$ 4.7×10^{11}

La condición matemática exacta de convergencia (necesaria y suficiente, a diferencia de la DD que solo es suficiente) es que el **radio espectral** de la matriz de iteración sea menor a 1. Para el "Orden 2" de arriba da 3.10 (Jacobi) y 8.35 (Gauss-Seidel) — muy por encima de 1, lo que confirma que ninguna semilla inicial converge (no es mala suerte con el punto de partida). No hace falta esto para resolver el ejercicio de la materia — alcanza con mostrar que no es DD y que la sucesión diverge en la práctica. Se deja acá solo como referencia.

Cómo encontrar un orden que sí converja

Si te dan una matriz que no converge tal como está ordenada, hay un truco rápido para encontrar un reordenamiento de filas que sí sea diagonalmente dominante, sin tener que probar todas las permutaciones a mano:

Para cada fila, fijate en qué columna está el coeficiente de **mayor valor absoluto** — ese es el mejor candidato para quedar en la diagonal de esa fila. Si asignás cada fila a la columna de su coeficiente más grande, y esa asignación no repite ninguna columna (cada fila "elige" una distinta), esa reordenación es DD.

EJEMPLO: SISTEMA (2) DE ARRIBA

Fila A: $-2x + y + 5z = 15 \rightarrow [-2, 1, 5] \rightarrow$ el mayor es 5 (col. z)

Fila B: $4x - 8y + z = -21 \rightarrow [4, -8, 1] \rightarrow$ el mayor es -8 (col. y)

Fila C: $4x - y + z = 7 \rightarrow [4, -1, 1] \rightarrow$ el mayor es 4 (col. x)

Sin conflicto (cada una eligio una columna distinta) $\rightarrow$ orden nuevo:

fila que quiere columna x primero (Fila C)

fila que quiere columna y segunda (Fila B)

fila que quiere columna z tercera (Fila A)

Eso da exactamente el "Orden 1" de arriba — el que ya sabíamos que convergía.

- Si **dos filas compiten** por la misma columna, no hay asignación directa — hay que probar a mano cuál cede esa columna y se queda con su segunda opción, y volver a chequear DD.
- El truco encuentra un orden DD, que es condición *suficiente* — puede existir un orden que converja sin ser DD, pero para los ejercicios de la materia, si existe un reordenamiento que sirva, este método normalmente lo encuentra directo.
- Si ninguna asignación logra que cada fila domine en su columna, es señal de que ese sistema no tiene ningún reordenamiento que lo vuelva DD (aunque podría converger

igual, sin garantía).

Caso puntual: un "no converge" ¿es real, o es demasiado lento?

El chequeo de divergencia del pseudocódigo (comparar el error contra el de la iteración anterior) a veces corta un caso que en el fondo *sí* converge — solo que muy lento, y sin bajar de forma perfectamente monótona.

EJEMPLO REAL

Matriz $\begin{bmatrix} 5 & 7 & 6 & 5 \\ 7 & 10 & 8 & 7 \\ 6 & 8 & 10 & 9 \\ 5 & 7 & 9 & 10 \end{bmatrix}$, $b = [23, 32, 33, 31]$ — ninguna fila es DD.

- **Jacobi:** diverge de verdad — el error crece feo desde el arranque (radio espectral de la matriz de iteración ≈ 2.48).
- **Gauss-Seidel:** con el algoritmo tal como se da en clase, corta en la iteración 31 con "no converge" — el error venía bajando de 4.6 a 0.0045 durante 30 iteraciones, y ahí tuvo un repunte mínimo (0.004536 $\rightarrow$ 0.004570) que dispara el corte.

Cómo distinguir uno del otro, mirando la salida:

- Si el error venía **creciendo feo** desde el principio (como Jacobi acá) $\rightarrow$ divergencia real, sin vuelta.
- Si el error venía **bajando de forma consistente** durante muchas iteraciones y recién al final tuvo un repunte chico (como Gauss-Seidel acá) $\rightarrow$ sospechoso de ser solo lento, no necesariamente divergente de verdad.
- Ante la duda, se puede correr una versión con el chequeo de divergencia relajado (que solo corte si el error realmente explota) y muchas más iteraciones de margen, para ver qué pasa en el largo plazo — sin usar esa versión como la respuesta a entregar, solo para entender qué está pasando.

EN ESTE CASO PUNTUAL

Dejándolo correr sin el corte temprano, Gauss-Seidel termina convergiendo — pero recién en la iteración **1276** (con $w=1$, tolerancia $1e-4$).

TRAMPA EXTRA: EL CRITERIO DE CORTE PUEDE MENTIR

En esa iteración 1276, el error "sucesivo" ($\|X_{k+1} - X_k\|$) daba 9.99×10^{-5} — ya por debajo de la tolerancia. Pero el error real contra la solución exacta era 0.032, más de

300 veces mayor. Esto pasa porque cuando la convergencia está casi estancada (radio espectral pegado a 1), dos iteraciones consecutivas se parecen mucho entre sí aunque las dos estén todavía lejos de la respuesta real — el criterio de corte mide cuánto cambió de un paso al otro, no cuánto falta para la solución exacta.

Criterio de corte por residuo

Alternativa a comparar dos iteraciones consecutivas: reescribiendo $Ax=b$ como $Ax-b=0$, se puede medir directamente qué tan bien x_k resuelve la ecuación real, en vez de cuánto se movió respecto al paso anterior:

$$\text{residuo}_k = \|A \cdot x_k - b\|$$

Ventaja sobre $\|x_{k+1} - x_k\|$: no lo puede engañar una racha de pasos chiquitos que no reflejan la distancia real a la respuesta — justo la trampa de arriba. Se implementa igual que antes, solo cambiando qué se calcula como "error" en cada iteración (y qué se compara contra la tolerancia).

VERIFICADO: PROBLEMA 2 RESUELTO DE NUEVO CON ESTE CRITERIO

Sistema a) (bien condicionado): prácticamente lo mismo que antes — Jacobi 48 iteraciones (antes 47), Gauss-Seidel 14 (igual). Mismo resultado $x=(1,1,1)$.

Sistema b) (el caso límite de arriba): sigue dando "no converge" para los dos métodos con el algoritmo dado — el criterio de residuo no lo arregla mágicamente, sigue siendo el mismo sistema límite.

Donde se ve la ventaja de verdad: dejando correr el sistema b) más allá del corte, comparando los dos criterios a la **misma tolerancia numérica** ($1e-4$):

Criterio	termina en iter	error real vs. solución e
Diferencia sucesiva	1276	0.032
Residuo $\ Ax-b\ $	1750	0.0074 (~4x más pr

A la misma tolerancia, el criterio de residuo tarda más en darse por satisfecho, pero cuando lo hace el resultado es varias veces más preciso — porque exige que la ecuación esté realmente resuelta, no solo que el último paso haya sido chico.

Sistemas de banda: aprovechar la estructura

Cuando una matriz tiene muchos ceros lejos de la diagonal (estructura de banda), no hace falta recorrer las n columnas en cada fila durante la iteración — alcanza con mirar la banda.

ANCHO DE BANDA

$$p = \max\{ |i-j| \} \text{ para todo } (i,j) \text{ con } A[i][j] \neq 0$$

Se calcula recorriendo toda la matriz una sola vez — es un algoritmo general, sirve para cualquier matriz, no hay que asumir nada de su estructura de antemano. Encontrado p , en Gauss-Seidel cada fila i recorre solo $j \in [i-p, i+p]$ en vez de $j \in [0, n-1]$ — fuera de ese rango ya se sabe que es cero, ni vale la pena mirarlo.

VERIFICADO: SISTEMA PENTADIAGONAL 50×50

Diagonal 12, offset ± 1 -2, offset ± 2 1 (ancho de banda real = 2, detectado automáticamente). Mismo resultado con las dos versiones (12 iteraciones, misma solución) — la diferencia es solo velocidad:

Denso: 12.10 us por resolución (recorre las 50 columnas por fila)
Banda: 4.37 us por resolución (recorre ~5 columnas por fila)
Speedup: 2.77x

Con $n=50$ la diferencia ya se nota. Crece mucho más para n grande: el denso escala con n^2 por iteración, el de banda con $n \cdot p$ — la ventaja del ancho de banda se vuelve enorme cuanto más grande el sistema (y más angosta la banda relativa a n).

GENERAR SISTEMAS GRANDES: PROGRAMÁTICAMENTE, NO A MANO

Un sistema de 50×50 (o más) no se puede tipear a mano en un archivo de texto. Conviene escribir un programita en C++ que arme el patrón (diagonal, offsets, condiciones de borde en las primeras/últimas filas) con un loop, y lo escriba a un archivo con el mismo formato $[A | b]$ que ya se lee siempre — así el sistema de prueba queda reproducible y es parte de la propia entrega, no algo hecho aparte a mano.

Relajación en la práctica: comparación real de ω

Caso real: sistema 6×6 tridiagonal ampliado (bandas hasta ± 2), **no** diagonalmente dominante (4 de las 6 filas fallan la DD) — y aun así Gauss-Seidel converge sin problemas. Es la prueba concreta de que DD es condición suficiente, no necesaria.

VERIFICADO

$w = 1,0$ (Gauss-Seidel sin relajacion): 43 iteraciones
 $w = 1,1$ (sobrerelajacion): 39 iteraciones (~9% menos)

Misma solución en los dos casos: $x = (2, 0, 4, 4, 0, 2)$. Con un sistema que ya converge razonablemente bien, un w levemente mayor a 1 acelera un poco sin desestabilizar.

DETALLE DE BORDE, SIN IMPORTANCIA PRÁCTICA

En este sistema, dos filas tienen la diagonal **igual** (no mayor) a la suma de las demás — no cumplen la DD estricta tampoco, pero como el chequeo de código usa $<$ estricto, no se marcan como "no DD". No cambia el resultado del ejercicio, es solo una curiosidad del caso límite exacto.

CAPÍTULO 3 — INTERPOLACIÓN

Idea general

Dado un conjunto de $n+1$ datos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ con los x_i todos distintos, existe **uno y solo un** polinomio de grado $\leq n$ que pasa exactamente por todos ellos. Sirve para estimar un valor intermedio (un y para un x que no está en la tabla de datos) sin conocer la función real detrás de los datos.

FORMA GENERAL DEL POLINOMIO DE GRADO N

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

Dos formas de conseguirlo: Lagrange (evalúa directo, sin armar el polinomio) y coeficientes indeterminados (arma el polinomio completo, resolviendo un sistema lineal — ya sabemos

hacer esto).

Lagrange

$$P_n(x) = \sum_{k=0}^n y_k \cdot C_{nk}(x) \quad \text{con} \quad C_{nk}(x) = \prod_{i \neq k} \frac{x - x_i}{x_k - x_i}$$

Cada $C_{nk}(x)$ ("cardinal" de Lagrange) tiene una propiedad clave que hace que todo funcione:

$$C_{nk}(x_k) = 1 \quad C_{nk}(x_j) = 0 \quad (\text{para } j \neq k)$$

Por eso, al evaluar en cualquiera de los puntos originales x_j , todos los términos de la suma se anulan menos uno, y queda $P_n(x_j) = y_j \cdot 1 = y_j$ — el polinomio pasa exacto por los datos, por construcción.

PSEUDOCÓDIGO (EVALÚA P_n EN UN PUNTO $\hat{x}$, NO ARMA EL POLINOMIO)

```
Leer datos y guardarlos: x[n+1], y[n+1]
Ingresar x_hat
suma = 0
Do k = 0, ..., n
  prod = 1
  Do i = 0, ..., n
    if (i != k)
      prod = prod * (x_hat - x[i]) / (x[k] - x[i])
    end if
  end Do
  suma = suma + y[k]*prod
end Do
Mostrar resultado: suma
```

DESVENTAJA

Lagrange solo devuelve el **valor** interpolado en un punto puntual — no arma la forma funcional del polinomio (los coeficientes $a_0, a_1, \dots$). Si hace falta el polinomio completo (por ejemplo, para derivarlo o graficarlo), conviene el método de coeficientes indeterminados.

Coeficientes indeterminados (Vandermonde)

Cada punto (x_i, y_i) da una ecuación lineal en las incógnitas $a_0, \dots, a_n$: $a_0 + a_1 x_i + a_2 x_i^2 + \dots + a_n x_i^n = y_i$. Con $n+1$ puntos quedan $n+1$ ecuaciones para $n+1$ incógnitas — un sistema lineal $Ax=b$ (la misma matriz de Vandermonde que ya vimos en [interpolación polinómica](#) de Eliminación gaussiana), que se resuelve con Gauss + pivoteo, tal como ya lo tenemos hecho.

PSEUDOCÓDIGO

Leer los datos y guardarlos: $x[n+1], y[n+1]$

Armar A y b:

```
Do i = 0, ..., n
  Do j = 0, ..., n
    A[i][j] = pow(x[i], j)
  end Do
  b[i] = y[i]
end Do
```

Resolver el sistema: call GaussPivot(A, b) -> devuelve los a_i

Si evaluamos en x_{hat} :

```
suma = 0
Do i = 0, ..., n
  suma = suma + a[i]*pow(x_hat, i)
end Do
Mostrar resultado: suma
```

Cómo verificar que el resultado está bien

1. **El chequeo básico, siempre válido:** evaluar el polinomio en los puntos *originales* y confirmar que $P_n(x_i) = y_i$ exacto, para todos los i . Si no da exacto ahí, hay un error en el cálculo — no hace falta ni conocer la respuesta de antemano.
2. **Comparar los dos métodos entre sí.** Lagrange y coeficientes indeterminados construyen matemáticamente *el mismo* polinomio (por unicidad), así que evaluados en el mismo punto tienen que dar exactamente igual. Si difieren, hay un bug en alguno de los dos.
3. **¿El valor es razonable?** — con cuidado. Para polinomios de grado bajo y datos que no cambian mucho, el valor interpolado suele caer entre los y vecinos. Pero no es una regla general: un polinomio de grado alto puede oscilar entre los puntos (fenómeno de Runge) y dar valores fuera de ese rango sin que sea un error. Sirve como señal de alerta, no como prueba.
4. **Interpolador vs. extrapolar.** Evaluar *dentro* del rango $[x_0, x_n]$ (interpolador) es razonablemente confiable. Evaluar *afuera* de ese rango (extrapolar) es mucho menos confiable — nada garantiza que el polinomio se siga pareciendo a los datos reales más allá de donde fue ajustado.

EJEMPLO VERIFICADO

Puntos $(0,1)$, $(3,2)$, $(6,8)$ → polinomio de grado 2.

$$P_2(x) = 1 - 0,5x + 0,277778x^2$$

Chequeo en los puntos originales: $P_2(0)=1$ ✓, $P_2(3)=2$ ✓, $P_2(6)=8$ ✓.

Interpolando en $x=2$ (adentro del rango): Lagrange y coeficientes indeterminados dan los dos $P(2) = 1,1111 (=10/9)$. Coincide.

Interpolando en $x=4$: $P(4) = 3,4444 (=31/9)$ — cae entre $y=2$ (en $x=3$) y $y=8$ (en $x=6$), como se esperaba para este caso de grado bajo.

Interpolación fuerza al polinomio a pasar **exacto** por cada punto. Eso funciona bien con datos limpios, pero si los datos vienen de mediciones con error apreciable, forzar el paso exacto por cada uno (ruido incluido) puede dar un polinomio que oscila mucho y no sirve para predecir nada — sobre todo si n es grande. A veces, con solo mirar los datos ("por inspección"), ya se intuye un comportamiento simple (lineal, por ejemplo) escondido detrás del ruido.

Regresión no busca pasar exacto por los puntos — busca la función que quede *lo más cerca posible* de todos ellos a la vez, aceptando algo de error en cada uno a cambio de una forma simple y estable.

Regresión lineal

El caso más simple: buscar $F(x) = ax + b$ tal que el error cuadrático total sea mínimo. Con $e_i = |y_i - F(x_i)|$, el "criterio de cuadrados mínimos" busca minimizar:

$$S_T(a,b) = \sum_{i=0}^n (y_i - ax_i - b)^2$$

Derivando e igualando a 0 ($\partial S_T / \partial a = 0$, $\partial S_T / \partial b = 0$) se llega a un sistema lineal 2×2 para las incógnitas a, b :

$$\frac{\sum x_i^2}{\sum x_i} \frac{a}{b} = \frac{\sum x_i y_i}{\sum y_i}$$

EJEMPLO VERIFICADO

x : 0, 1, 1.5, 3, 5 y : 1.2, 2.7, 3.9, 7.1, 10 (5 datos)

$$\begin{array}{ll} \text{Sum } x^2 = 37.25 & \text{Sum } x = 10.5 \\ \text{Sum } xy = 79.85 & \text{Sum } y = 24.9 \end{array}$$

$$\begin{bmatrix} 37.25 & 10.5 \\ 10.5 & 5 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 79.85 \\ 24.9 \end{bmatrix}$$

$$a = 1.8132 \quad b = 1.1724$$

$F(x) = 1,81x + 1,17$. Para corroborar: evaluar F en cada x y ver si da parecido a las y reales (no exacto, pero cerca).

Coeficiente de correlación

¿Cómo saber si el ajuste lineal es bueno, o si los datos en realidad no tienen ninguna relación lineal y son solo mediciones dispersas de un mismo valor? Se compara la dispersión total de los datos contra el error que deja el ajuste:

$$S_t = \sum (y_i - \bar{y})^2 \quad (\text{dispersión total, respecto al promedio}) \quad S_r = \sum (y_i - F(x_i))^2 \quad (\text{error que deja el ajuste})$$

$$r = \sqrt{(S_t - S_r) / S_t}$$

- Ajuste lineal perfecto $\rightarrow S_r=0 \rightarrow r=1$.
- $r=0 \Leftrightarrow S_t=S_r$: el ajuste no explica nada más de lo que ya explicaba la dispersión estadística de los datos — el "ajuste" no sirvió de nada.
- Cuanto más cerca de 1 esté r , mejor el ajuste (como referencia informal de tus apuntes: $r < 0,7$ ya se considera medio/malo).

APLICADO AL EJEMPLO ANTERIOR

$\bar{y} = 24,9/5 = 4,98$ $S_t = 50,35$ $S_r = 0,378 \rightarrow r = 0,99624$ — muy buen ajuste.

Regresión con funciones arbitrarias

La regresión por cuadrados mínimos no está limitada a rectas ni polinomios — sirve para **cualquier** función con parámetros a ajustar. La receta es siempre la misma: plantear S_T con la función propuesta, derivar respecto a cada parámetro, igualar a 0, y resolver el sistema lineal resultante (con Gauss, como siempre).

EJEMPLO REAL (P4 - PARCIAL 1, 2023)

Ajustar $F(x) = a \cdot \sin(x) + b \cdot \cos(x)$ a 7 puntos. Derivando S_T respecto a a y b :

$$\frac{\sum \sin^2 x_i}{\sum \cos x_i \sin x_i} \quad \frac{\sum \cos x_i \sin x_i}{\sum \cos^2 x_i} \quad \frac{a}{b} = \frac{\sum y_i \sin x_i}{\sum y_i \cos x_i}$$

Mismo sistema 2×2 de siempre, distintas sumas — resolver con Gauss. Con los 7 datos del parcial: $a=0,678$, $b=-1,928$, $r=0,970$ (buen ajuste, no perfecto).

Regresión polinómica (caso general)

Generaliza la regresión lineal a un polinomio de grado p cualquiera: $F(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_p x^p$. Hacen falta al menos $p+1$ puntos para que el sistema quede determinado. Derivando S_T respecto a cada a_k ($p+1$ derivadas) se llega a un sistema lineal $(p+1) \times (p+1)$, con esta forma general para cualquier p (las "ecuaciones normales"):

$$A_{\ell m} = \sum_{i=0}^n x_i^{\ell+m} \quad b_{\ell} = \sum_{i=0}^n y_i \cdot x_i^{\ell} \quad (\ell, m = 0, \dots, p)$$

Con $p=1$ esto se reduce exactamente a la matriz de regresión lineal de arriba — es el mismo caso, generalizado.

PSEUDOCÓDIGO

```
Ingresa los datos (x,y) -> guardar en x[n+1], y[n+1]
Ingresa p (grado del polinomio)
Si (n+1) < (p+1) -> imprimir "no hay suficientes datos", parar
```

```

Armar A, b: (A[p+1][p+1], b[p+1])
Do l = 0,...,p
    sumaAy = 0
    Do i = 0,...,n
        sumaAy = sumaAy + pow(x[i],l)*y[i]
    end Do
    b[l] = sumaAy

    Do m = 0,...,p
        sumaAx = 0
        Do i = 0,...,n
            sumaAx = sumaAx + pow(x[i], l+m)
        end Do
        A[l][m] = sumaAx
    end Do
end Do

Llamar a Eliminacion Gaussiana(A,b) -> devuelve los a_k
Mostrar resultados

Calcular el coeficiente de correlacion:
yprom = (suma de y[i]) / (n+1)
sumaT = 0, sumaR = 0
Do i = 0,...,n
    sumaT = sumaT + (y[i]-yprom)^2
    fi = suma_j( a[j]*pow(x[i],j) )    para j=0,...,p
    sumaR = sumaR + (y[i]-fi)^2
end Do
r = sqrt( (sumaT-sumaR)/sumaT )
Mostrar r

```

CÓDIGO VERIFICADO: REGRESIONPOLINOMICA.CPP

Implementa este pseudocódigo tal cual — con $p=1$ ES la regresión lineal. Dos chequeos:

- **Contra el ejemplo de regresión lineal de arriba** ($p=1$): dio $a_0=1,17237$, $a_1=1,81316$, $r=0,996245$ — exacto a lo calculado a mano.
- **Ajuste exacto sin ruido** ($p=2$, datos generados desde $y=2+3x-x^2$ sin error): recuperó $a_0=2$, $a_1=3$, $a_2=-1$ exactos, con $Sr=0$ y $r=1$ — cuando no hay ruido, la regresión coincide con la interpolación exacta.

Qué código usar para cada tipo de problema

Mapa de todos los .cpp del repo, agrupados por carpeta. Muchos se parecen entre sí a propósito — son variantes del mismo algoritmo con un solo cambio puntual (el criterio de corte, el valor inicial, si conviene o no la función exacta) — así que la tabla marca justo esa diferencia, que es lo que hay que mirar para elegir cuál.

LocalizacionRaices/

ARCHIVO	QUÉ HACE	CUÁNDO USARLO
raices.cpp	Menú con los 6 métodos de la materia: bisección, falsa posición, punto fijo, Newton-Raphson, secante, raíz n-ésima.	Base para cualquier ejercicio de raíces. Cada función tiene la $f(x)$ (y a veces la tolerancia/máx. iteraciones) hardcoded como <code>auto f = [] (double x) { ... }</code> al principio — para un problema nuevo hay que editar esa línea y recompilar, no sirve "tal cual" para dos ejercicios distintos a la vez.
newtonRaphsonProblema1.cpp	Newton-Raphson standalone, ya con la $f(x)$ de un problema puntual.	Cuando la función <code>newtonRaphson()</code> de <code>raices.cpp</code> ya está "ocupada" por otro ejercicio (con su propia $f(x)$ /tolerancia ya ajustadas) y no querés pisarlo — se copia este patrón a un archivo nuevo en vez de sobrecribir el que ya funciona para otro problema.

OJO ANTES DE REUSAR CUALQUIER MÉTODO DE RAÍCES.CPP

Al menos dos funciones quedaron con parámetros de un ejercicio específico, no genéricos: `falsaposicion()` tiene `iter<10` hardcoded y `newtonRaphson()` tiene `iter<4` — ambos porque un enunciado pidió justo el resultado en esa iteración puntual. Antes de correr cualquiera de estos para un problema nuevo, revisar la $f(x)$, la tolerancia y el `max_iter` de esa función particular.

Matrices/

ARCHIVO	QUÉ HACE	CUÁNDO USARLO
<code>eliminacionGausseana.cpp</code>	Menú con Gauss (triangulación + retrosustitución) y Gauss-Jordan. Calcula determinante (con signo, compensando swaps) y norma de Frobenius.	Cualquier sistema lineal a resolver de forma directa (no iterativa), sea cual sea su tamaño. También sirve como plantilla de Gauss para cuando otro programa necesita resolver un sistema aparte (interpolación por coeficientes indeterminados, ajuste de curvas) — en esos casos el mismo Gauss+pivoteo está copiado adentro del archivo correspondiente, no se hace <code>#include</code> de este.
<code>jacobi.cpp</code>	Iterativo, actualiza todo el vector con los valores de la iteración anterior. Corte por diferencia sucesiva $\ x_{\text{nuevo}} - x_{\text{viejo}}\ $.	Cuando el enunciado pide Jacobi explícitamente y el criterio de error es el "estándar" del curso (diferencia entre iteraciones, no residuo).
<code>jacobiResiduo.cpp</code>	Igual que <code>jacobi.cpp</code> , pero el corte es por residuo $\ A \cdot x - b\ $ en vez de diferencia sucesiva.	Solo cuando el enunciado pide ese criterio específico (ej. "hasta que el residuo sea menor a...").
<code>gaussSeidel.cpp</code>	Igual filosofía que Jacobi pero con actualización in-place (un solo array), más relajación w . Corte por diferencia sucesiva.	Cuando el enunciado pide Gauss-Seidel (con o sin relajación) y el criterio es diferencia sucesiva. El vector inicial arranca en 0 por defecto — si el enunciado da un x_0 distinto, hay que cambiar esa línea antes de correr.
<code>gaussSeidelResiduo.cpp</code>	Igual que <code>gaussSeidel.cpp</code> pero con corte por residuo $\ A \cdot x - b\ $.	Mismo caso que <code>jacobiResiduo.cpp</code> : solo si el enunciado pide residuo en vez de diferencia sucesiva.
<code>gaussSeidelMejorado.cpp</code>	Variante exploratoria: solo declara divergencia si el error realmente explota (no ante un repunte chico puntual), <code>max_iter</code> mucho más alto.	NO es para entregar — sirve para diagnosticar, aparte, si un "no converge" del algoritmo de clase es real o solo una convergencia muy lenta. La respuesta que se entrega siempre sale de <code>gaussSeidel.cpp</code> tal cual, con el pseudocódigo de la profesora.

ARCHIVO	QUÉ HACE	CUÁNDO USARLO
generarBanda.cpp + gaussSeidelBanda.cpp	El primero genera un sistema de banda (ej. pentadiagonal, n grande) y lo escribe a archivo. El segundo compara Gauss-Seidel "denso" (recorre todas las columnas) contra una versión que solo recorre el ancho de banda, y mide el speedup.	Ejercicios de sistemas grandes con estructura de banda, donde piden comparar tiempos de cómputo aprovechando (o no) esa estructura.

Interpolacion/

ARCHIVO	QUÉ HACE	CUÁNDO USARLO
lagrange.cpp	Evalúa $P_n(\hat{x})$ directo en un punto puntual, sin armar el polinomio expandido.	El enunciado da una tabla de puntos y solo pide el valor interpolado en uno o dos puntos concretos (no el polinomio en sí).
coeficientes.cpp	Vandermonde + Gauss: da los coeficientes $a_0 \dots a_n$ del polinomio expandido, y de paso lo evalúa en un punto.	El enunciado pide explícitamente "el polinomio de Lagrange" / sus coeficientes, no solo un valor puntual (ej. Problema 4, punto b).
lagrangeGrilla.cpp	Igual que lagrange.cpp pero evalúa en una grilla fina y exporta un CSV para graficar. Sin columna de error.	Los datos son experimentales/de tabla (no hay una $f(x)$ real conocida contra la cual comparar) y piden graficar la curva interpolada.
lagrangeFuncion.cpp	Se conoce la $f(x)$ real: genera nodos equiespaciados automáticamente, calcula error de interpolación y extrapolación, permite probar varios grados n en la misma corrida.	El enunciado da la función exacta y pide comparar el polinomio contra ella (ej. estudiar Runge, comparar grados, ver qué tan mal extrapola).
lagrangeNodosDados.cpp	Como lagrangeFuncion.cpp (se conoce $f(x)$, calcula error exacto), pero los nodos	Se conoce la $f(x)$ real, pero los nodos son una

ARCHIVO	QUÉ HACE	CUÁNDO USARLO
	vienen dados puntualmente en un archivo (no equiespaciados). Cubre en una corrida: armar el polinomio, evaluar+error en un punto, y exportar la grilla de error para graficar.	tabla específica del enunciado, no equiespaciados a elección.

AjusteDeCurvas/

ARCHIVO	QUÉ HACE	CUÁNDO USARLO
<code>regresionPolinmica.cpp</code>	Ajuste $F(x) = a_0 + a_1x + \dots + a_px^p$ (con $p=1$, regresión lineal simple). Arma y resuelve las ecuaciones normales, da S_t , S_r , r y el error cuadrático medio.	El modelo a ajustar es un polinomio en x (incluida la recta).
<code>regresionFuncionesBase.cpp</code>	Generaliza a $F(x) = a_0 \cdot \phi_0(x) + a_1 \cdot \phi_1(x) + \dots$ con cualquier combinación de funciones base. Arma A y b automáticamente a partir de la lista de ϕ , y resuelve el sistema él mismo.	El modelo NO es un polinomio en x (ej. $a \cdot \sin(x) + b \cdot \cos(x)$ del Problema 3) y querés que el programa arme y resuelva todo de punta a punta.
<code>evaluarAjuste.cpp</code>	Complemento de <code>regresionFuncionesBase.cpp</code> : NO arma ni resuelve ningún sistema — solo recibe a_0 , a_1 ya calculados (a mano, o resolviendo el sistema aparte con <code>eliminacionGausseana.cpp</code>) y evalúa $F(x)$ contra los puntos para dar S_t , S_r , r .	El enunciado pide explícitamente plantear el sistema a mano y resolverlo con Gauss (mostrando esa cuenta), y recién después evaluar el ajuste — separa en dos pasos lo que <code>regresionFuncionesBase.cpp</code> hace en uno solo.

EL PATRÓN QUE SE REPITE EN CASI TODOS

La gran mayoría de estos programas comparten el mismo esqueleto (leer archivo de puntos/matriz, armar el sistema, Gauss con pivoteo parcial) y solo cambian en **una línea marcada explícitamente** con un comentario tipo `// ACA se elige el problema — la  $f(x)$ , las funciones base  $\phi$ , o el criterio de corte`. Antes de correr cualquiera de estos para un ejercicio nuevo, ubicar esa línea puntual y confirmar que

coincide con el enunciado, en vez de asumir que "ya está bien" porque compiló y corrió sin errores.

TRANSVERSAL – TIPOS DE ERROR

Estimado vs. exacto, absoluto vs. porcentual

Estos cuatro nombres se mezclan todo el tiempo en los enunciados, y en realidad son dos ejes independientes que se cruzan. El criterio es el mismo para bisección, falsa posición, punto fijo, Newton-Raphson, secante, Jacobi y Gauss-Seidel — solo cambia la fórmula puntual de cada método.

EJE 1 – ¿CONTRA QUÉ SE COMPARA?

- **Estimado:** compara la iteración actual contra la **iteración anterior** (x_{nuevo} vs x_{viejo}). Es el único que un programa real puede calcular — nunca conoce la solución exacta — y es lo que se usa como criterio de corte del `while`.
- **Exacto:** compara la aproximación actual contra la **solución analítica conocida** (x_{aprox} vs x_{exacto}). Solo se puede calcular en ejercicios donde el enunciado permite obtener la raíz real a mano (el punto "a" del parcial modelo, por ejemplo) — sirve para *verificar* qué tan buena es la estimación, no como criterio de corte del programa.

EJE 2 – ¿ABSOLUTO O PORCENTUAL?

- **Absoluto:** la diferencia cruda, sin normalizar. $\text{error} = |A - B|$.
- **Porcentual:** el mismo valor absoluto, dividido por algo de referencia y $\times 100$ para expresarlo como %.

Las cuatro combinaciones

ESTIMADO – ABSOLUTO

$$\text{error} = |x_{\text{nuevo}} - x_{\text{viejo}}|$$

ESTIMADO – PORCENTUAL (FÓRMULA (#) DEL PARCIAL MODELO, PARA BISECCIÓN/NEWTON-RAPHSON)

$$\varepsilon = \frac{|x_{\text{nuevo}} - x_{\text{viejo}}|}{\frac{1}{2} \cdot |x_{\text{nuevo}} + x_{\text{viejo}}|} \times 100$$

EXACTO – ABSOLUTO

$$\text{error}_{\text{exacto}} = |x_{\text{aprox}} - x_{\text{exacto}}|$$

EXACTO – PORCENTUAL

$$\varepsilon_{\text{exacto}} = \frac{|x_{\text{aprox}} - x_{\text{exacto}}|}{|x_{\text{exacto}}|} \times 100$$

LA FÓRMULA DEL "ESTIMADO" NO ES SIEMPRE LA MISMA

El denominador de la versión porcentual del error estimado **depende del enunciado** — no hay una única fórmula universal. Ejemplos ya vistos en la materia:

- Bisección/Newton-Raphson (parcial modelo): divide por el **promedio** de x_{nuevo} y x_{viejo} (fórmula (#) de arriba).
- Jacobi/Gauss-Seidel (ver [capítulo 2](#)): divide por la **norma del vector nuevo**, $E = \|X_{\text{nuevo}} - X_{\text{viejo}}\| / \|X_{\text{nuevo}}\| \times 100$.

Antes de programar el criterio de error, siempre conviene releer qué fórmula puntual pide *ese* enunciado — no asumir que es la misma del ejercicio anterior.

Ejemplo resuelto (Problema 1, parcial modelo)

$f(x) = (x+1)/(x+4) - 0.25x$, raíz exacta $x=2$ (punto a, despejado a mano).

Bisección en $[1.75, 3.2]$, última iteración (la 13ª):

```
x_aprox (c)           = 2.0001037598
error estimado (#)    = 0.0088492470 %   (contra la iteracion anterior)
```

```
error absoluto exacto    = 0.0001037598    (contra x=2)
error porcentual exacto  = 0.0051880 %    (contra x=2)
```

Newton-Raphson desde $x_0=1.0$, primera iteración:

```
x1                        = 2.1538461538
error estimado (#)       = 73.1707317073 % (contra  $x_0=1.0$ )
error absoluto exacto    = 0.1538461538    (contra x=2)
error porcentual exacto  = 7.6923076923 %  (contra x=2)
```

POR QUÉ VALE LA PENA CALCULAR LOS DOS

El error **estimado** es el único que existe en un problema real (nunca conocés la raíz exacta de antemano) — es la garantía práctica de que "ya convergió lo suficiente". El error **exacto** solo se puede calcular en ejercicios de práctica, y sirve para confirmar que el estimado no está mintiendo: en el ejemplo de Newton-Raphson de arriba, el error estimado (73%) es mucho más pesimista que el exacto (7.7%) en la primera iteración — son cantidades distintas, no hay por qué esperar que coincidan, sobre todo lejos de la raíz.

OJO CON QUÉ PIDE CADA ENUNCIADO, PALABRA POR PALABRA

No todos los ejercicios piden las cuatro variantes. Leer con cuidado si dice "error", "error absoluto", "error porcentual" o "error exacto" a secas — cada palabra extra cambia cuál de las cuatro fórmulas de arriba corresponde. Si el enunciado dice "error exacto" sin la palabra "porcentual", es la versión **absoluta** (resta simple, sin dividir ni multiplicar por 100) — no asumir automáticamente que es un porcentaje.

TRANSVERSAL — ERRORES RECURRENTES EN C++

Cosas que se repitieron en más de un método, vale la pena tenerlas de checklist antes de entregar cualquier ejercicio:

1. **1-indexado vs 0-indexado.** El pseudocódigo de clase arranca los índices en 1; los arrays de C++ arrancan en 0. Revisar siempre el *punto de arranque* de cada loop (no solo el

límite superior).

2. **&&, no ||, en la condición de corte.** "Seguir iterando mientras no convergió **y** no me pasé del máximo de iteraciones" es una condición con &&. Con || el tope de iteraciones deja de funcionar como salvaguarda.
3. **Inicializar las variables de "valor anterior"** (c_viejo, x_viejo) antes de usarlas por primera vez — si no, el primer error calculado es basura de memoria.
4. **Guardas de división por cero** antes de dividir: el pivote en Gauss/Jacobi, la derivada en Newton, el denominador en secante.
5. **Guardar valores absolutos donde corresponde** (fabs(. . .)) — comparar o acumular un valor con signo cuando la lógica pide magnitud es una fuente de bugs difícil de ver a simple vista.
6. **cout trunca a 6 cifras significativas por default** — si la tolerancia es más estricta que eso (ej. tol=1e-12), el cálculo interno ya tiene esa precisión, pero no se ve en pantalla a menos que agregues `#include <iomanip>` y `cout << fixed << setprecision(N)` antes de imprimir. Ojo con el sentido inverso: si la tolerancia es floja (ej. 1e-5 o 1e-6), **no** forzar más decimales en el print — mostrarías dígitos que el cálculo no garantiza como correctos. La precisión que se imprime tiene que ser consistente con la tolerancia que realmente se usó para calcular, ni de más ni de menos.
7. **Un bug en una rama de código que nunca se ejecuta no se nota** — el ejemplo real: el signo del determinante en Gauss estaba mal (faltaba compensar los swaps de fila del pivoteo), pero como ningún sistema anterior necesitaba pivotear de verdad, esa rama nunca corrió y el bug quedó escondido varios ejercicios. Se destapó recién con un caso que *forzaba* el pivoteo. Vale la pena, al probar un programa, buscar a propósito un caso que dispare cada rama del código (pivoteo, matriz singular, semilla que hace fallar el chequeo de derivada, etc.), no solo el caso fácil que "anda a la primera".